

# **LIVE EVENT DETECTION FOR PEOPLE'S SAFETY USING NLP AND DEEP LEARNING**

**A SEMINAR REPORT**

submitted by

**SOLAMAN AJI  
MBI21CS056**

to

the APJ Abdul kalam Technological University  
in partial fulfillment for the award of the degree  
of

**Bachelor of Technology**

In

*Computer Science and Engineering*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**MAR BASELIOS INSTITUTE OF TECHNOLOGY & SCIENCE  
NELLIMATTOM, KERALA, INDIA -686693**

**December, 2024**

# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## MAR BASELIOS INSTITUTE OF TECHNOLOGY AND SCIENCE NELLIMATTOM



### CERTIFICATE

This is to certify that the seminar report entitled **“LIVE EVENT DETECTION FOR PEOPLE’S SAFETY USING NLP AND DEEP LEARNING”** Submitted by **SOLAMAN AJI** with Reg no: **MBI21CS056** during the academic year 2024-2025 towards the partial fulfillment of the requirement for the award of Bachelor of Technology in Computer Science & Engineering of APJ Abdul Kalam Technological University Trivandrum, Kerala is a bonafide work carried out by him in this department under our guidance and supervision.

Mrs.Nisreen M Ali  
Asst. Prof. CSE Dept  
Seminar Guide

Mrs.Mintu Thomas  
Asst. Prof. CSE Dept  
Seminar coordinator

Mr. Midhun Mathew  
Head of Department  
Computer Science &  
Engineering

## ACKNOWLEDGEMENT

First and the foremost, I thank **ALMIGHTY GOD** who gave me the inner strength, resource and ability to complete my work successfully, without which all my efforts would have been in vain.

I express my sincere gratitude and thanks to **Dr.Thomas George**, our Principal and Asst. **Prof. Midhun Mathew**, Head of the Department, Computer Science and Engineering, for providing the facilities and all the encouragement and support.

I am obliged to my guide and seminar supervisor **Asst. Prof. Nisreen M Ali**, in Computer Science and Engineering Department, for her valuable guidance, inspiration and intellectual support extended to me for the successful completion of the seminar.

I express my sincere thanks to all the staff members in the Computer Science and Engineering who have taken sincere efforts in helping me to complete the seminar work.

Finally, I would like to acknowledge the heartfelt efforts, comments, criticisms, cooperation and tremendous support given to me by my dear friends and family members during the seminar without which this work would have been all the more difficult to accomplish.

## ABSTRACT

In the physical world, every event is accompanied by a unique sound signature, making it possible to analyze ambient noise to determine whether it signifies a positive or negative occurrence. Criminal activities such as robbery, assault, or homicide often produce distinctive sounds that, if detected early, could facilitate timely intervention and potentially save lives. Additionally, many conventional systems require direct human intervention, making them less effective in situations where immediate response is critical. A more advanced and efficient approach involves leveraging Natural Language Processing (NLP) and deep learning techniques to analyze sounds from a person's surroundings in real time. By continuously monitoring ambient noise through a live recording module, the system processes the captured sounds using a prediction module to classify them as safe or dangerous. If a potential threat is detected, an automatic alert message—along with the person's precise geographical location—is instantly sent to emergency services and registered contacts via WhatsApp, SMS, or email, ensuring a rapid response to potentially life-threatening situations.

To ensure accurate threat detection, the system undergoes extensive training and testing using a Kaggle dataset containing various environmental sounds, including gunshots, screams, glass breaking, and other potential indicators of danger. Three deep learning models—Long Short-Term Memory (LSTM), 1D Convolutional Neural Network (1D CNN), and 2D Convolutional Neural Network (2D CNN)—are employed, each specializing in different aspects of audio classification and achieving an impressive average accuracy of 96.03%. The system offers flexibility by allowing users to choose between these models based on specific use cases, with LSTM excelling in analyzing sequential sound patterns and CNN models effectively capturing spatial features of audio spectrograms. By integrating deep learning with advanced audio signal processing, this solution provides a practical and scalable means of enhancing personal safety, particularly in high-risk scenarios where individuals may be vulnerable to physical harm. This work represents a significant step toward developing intelligent, automated security solutions that can proactively detect and respond to threats in real time, offering a promising approach to improving public and personal safety through sound-based event detection.

# CONTENTS

|                                          |    |
|------------------------------------------|----|
| AKNOWLEDGEMENT                           | i  |
| ABSTRACT                                 | ii |
| LIST OF FIGURES                          | iv |
|                                          |    |
| ABRREVATIONS                             | v  |
| 1. INTRODUCTION                          | 1  |
| 2. LITERATURE SURVEY                     | 2  |
| 2.1 BACKGROUND AND RELATED WORK          |    |
| 3. METHODOLOGY                           | 3  |
| 3.1 PROPOSED APPROACH                    |    |
| 3.1.1 SOFTWARE OVERVIEW                  | 3  |
| 3.1.2 OBJECTIVES                         | 3  |
| 3.1.3 INPUT AUDIO CLASSIFICATION         | 4  |
| 3.1.4 DATASET                            | 5  |
| 3.1.5 EXPLORATORY DATA ANALYTICS         | 6  |
| 3.1.5.1 SAMPLING                         | 6  |
| 3.1.5.2 CONVERTING INTO FREQUENCY DOMAIN | 7  |
| 3.1.5.3 MEL SPECTROGRAM REPRESENTATION   | 8  |
| 3.1.5.4 CLEANING OF THE SOUND DATA       | 9  |
| 4. EVALUATION                            |    |
| 4.1 PERFORMANCE OF DEEP LEARNING MODELS  | 15 |
| 4.1.1 CONFUSION MATRIX                   | 15 |
| 5. CONCLUSION                            | 17 |
| 6. REFERENCES                            | 18 |

## LIST OF FIGURES

| Fig No. | Title                                                         | Page No. |
|---------|---------------------------------------------------------------|----------|
| 1       | Training the dataset using ML models for live event detection | 4        |
| 2       | Audio signals in the time-domain for one sample audio         | 5        |
| 3       | Gunshot time-domain and frequency-domain                      | 7        |
| 4       | Decibel spectrogram and Mel spectrogram                       | 9        |
| 5       | Signal Envelope                                               | 10       |
| 6       | 1D-CNN                                                        | 11       |
| 7       | 2D-CNN                                                        | 12       |
| 8       | LSTM                                                          | 13       |
| 9       | Confusion Matrix                                              | 16       |

## **ABBREVIATIONS**

|      |                              |
|------|------------------------------|
| CNN  | Convolution Neural Network   |
| LSTM | Long Short-Term Memory       |
| NLP  | Natural Language Processing  |
| ARM  | Advanced RISC Machine        |
| EDA  | Exploratory Data Analytics   |
| GPU  | Graphical Processing Unit    |
| STFT | Short Time Fourier Transform |
| FFT  | Fast Fourier Transform       |





# CHAPTER 1

## INTRODUCTION

Today, humans pose the greatest threat to society by getting involved in robbery, assault, or homicide activities. Such circumstances threaten the people working alone at night in remote areas especially women. Any such kind of threat in real time is always associated with a sound/noise which may be used for an early detection. A system capable of detecting and categorizing these sounds could help identify the nature of ambient noise in real-time. While such applications are not yet commonplace, the technology to analyse sounds and text through natural language processing is already available and used in various fields. Numerous existing measures are available but none of them sounds efficient due to lack of accuracy, delays in exact prediction of threat.

The solution to this problem utilizes sound-based NLP techniques, incorporating a live recording module and a prediction module to analyze ambient sounds. The live recording module continuously captures sounds from a person's surroundings and transmits them to the prediction module, which classifies the sounds using deep learning models—LSTM, 1D CNN, and 2D CNN—trained on a Kaggle dataset for accurate predictions. By analyzing sound patterns, this system can differentiate between common sounds and those indicating potential danger, allowing for a proactive approach to safety. A significant benefit of this system is its reliance on a smartphone, eliminating the need for additional external hardware or wearable devices, making it convenient for users who only need to carry their phones. If the prediction module identifies a dangerous sound, such as a scream or crash, the system immediately and automatically sends an alert to emergency services, ensuring a rapid response to emergencies. Additionally, the system's adaptability allows it to function in various environments, from urban areas to remote locations, making it highly versatile. With further advancements, this approach could be integrated with existing smart city infrastructure, enhancing public safety on a larger scale.

## CHAPTER 2

### LITERATURE SURVEY

#### 2.1 BACKGROUND AND RELATED WORK

Sound classification has evolved from applications in speech and music analysis to more advanced uses in safety and security, where it can help detect and respond to emergencies. Early approaches involved using machine learning algorithms on audio data to recognize alarming sounds, such as screams, as part of emergency response systems. These systems, often based on hardware like Raspberry Pi or ARM controllers, aimed to provide real-time notifications in unsafe situations but were limited by bulkiness, making them impractical for personal, everyday use.

With the advent of deep learning, new methods have emerged for classifying sounds more accurately in noisy, dynamic environments. For instance, convolutional neural networks (CNNs) have shown promise in recognizing urban sounds, which could include potential threats like gunshots, breaking glass, or distress calls. Additionally, emotion recognition through speech analysis has been explored for detecting distress in voices, providing indirect indicators of danger. Although effective for identifying emotional states or distinguishing environmental sounds, these systems often struggle with recognizing physical threats in real-world, individual-level scenarios.

Other research focuses on unconventional applications, such as distinguishing drone sounds to monitor urban airspaces or classifying sounds within autonomous vehicles to detect passenger altercations. However, these systems remain limited to specific environments and lack adaptability for broader, real-time personal safety use cases. The challenge lies in creating user-friendly, portable systems that can effectively classify sounds in diverse settings, without relying on bulky hardware, to address the everyday safety needs of individuals.

## **CHAPTER 3**

### **METHODOLOGY**

#### **3.1 PROPOSED APPROACH**

##### **3.1.1 SOFTWARE OVERVIEW**

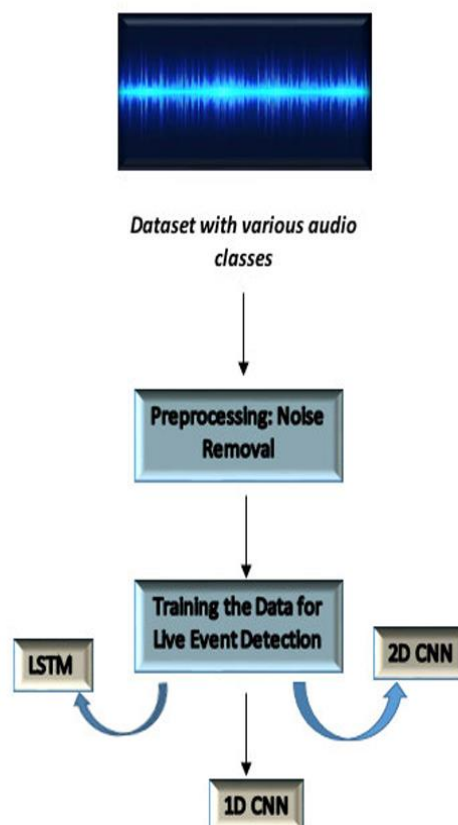
The software for this system operates as a mobile application, leveraging the device's built-in microphone to continuously monitor and classify audio signals in real time. It employs machine learning models, including LSTM, 1D CNN, and 2D CNN, trained on a specialized dataset to recognize sounds associated with potential danger, such as screams or gunshots. Upon detecting a suspicious audio pattern, the application immediately triggers an alert, sending the victim's location to emergency contacts and local authorities to ensure timely intervention.

##### **3.1.2 OBJECTIVES**

The goal of this system is to provide a real-time audio detection and classification tool that enhances personal safety by identifying and responding to potentially dangerous audio events through a smartphone application. By leveraging the device's built-in microphone, the system aims to detect suspicious sounds, such as screams, gunshots, or glass breaking, with high accuracy, minimizing false detections. Upon detecting such sounds, the application automatically shares the user's geographical location with designated emergency contacts and nearby law enforcement, enabling rapid response and increasing the likelihood of timely intervention. The objectives include achieving robust classification accuracy through the use of machine learning models trained on a diverse dataset of environmental sounds, as well as ensuring that the system is both accessible and effective on standard smartphone hardware without requiring specialized equipment. Through these features, the application aims to empower users by providing a reliable safety tool that works seamlessly in everyday environments.

### 3.1.3 INPUT AUDIO CLASSIFICATION

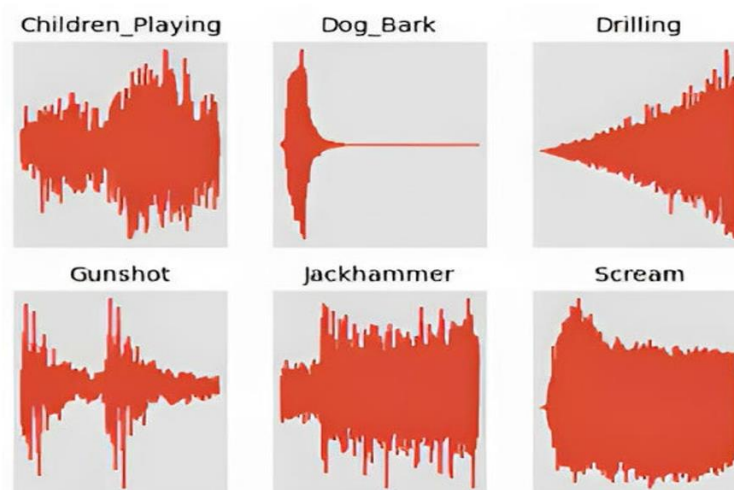
The main objective of the proposed system is to detect and classify the victim's live audio signals for immediate rescue. The system is intended to deliver its excellence as an application in any smartphone and it uses the default microphone configuration of it. On detection of suspicious audio patterns from the live input audio from microphone, the geographical location of the victim is shared to the emergency contacts in the phone as well as to the police patrol. The drawbacks inferred from the current violence detection scenarios related to audio event detection and classification accuracy are addressed for effective functioning which plays a vital role in avoiding false event classifications meanwhile ensuring the victim's safety through high classification accuracy. To carry out accurate prediction, training and testing of Kaggle dataset is carried out in 3 machine learning models namely LSTM, 1D CNN and 2D-CNN which is illustrated in figure 2 below:



**FIGURE 2.** Training the dataset using ML models for live event detection.

### 3.1.4 DATASET

The audio dataset used in this work consists of 13 classes (types) of audio signals, namely air conditioner, car horn, children playing, dog bark, drilling, engine idling, fire crackling, glass breaking, gunshot, jackhammer, scream, siren, and street music. Of these 13 classes, fire crackling, glass breaking, gunshot, and screaming are identified as audio types related to a potentially dangerous environment. The dataset described here is a customized dataset and is built by taking into account and combining parts of data from three different audio datasets in order to meet the requirements of the problem statement under research. The three datasets mentioned here are the UrbanSound8K dataset, which is a dataset with audio data for 50 different environmental sounds, and which is a dataset for the 'screaming' noise as this sound type is a crucial one for the requirement of the project, which focuses on the detection of threat from the ambient noise. Although the three original datasets from which the dataset for this project has been built, have over 50 different classes of audio signals, only 13 are kept for the final analysis as it would take less computational time for only 13 classes to be trained on the three deep learning models built during the course of this project, as compared to the larger size of a dataset with over 50 classes, taking into account the fact that this research focuses more on the successful development of a working prototype than a production-level software.



**FIGURE 3.** The audio signals in the time domain for one sample audio each from 6 of the 13 classes considered in the research.

### 3.1.5 EXPLORATORY DATA ANALYTICS (EDA)

Exploratory Data Analysis (EDA) in audio signal processing is a crucial step in understanding and preparing audio data before applying machine learning or deep learning models. By examining the data, EDA helps identify patterns, detect anomalies, and ensure data quality, which improves the accuracy and performance of predictive models. Key stages in EDA for audio data include sampling, where a subset of data is taken to represent the whole; converting time-domain signals to frequency domain, which reveals the sound's frequency components creating Mel spectrograms, which capture the audio's time-frequency information in a format that resembles human auditory perception; and cleaning the sound data to remove noise or irrelevant information. These steps facilitate deeper insights into the structure and characteristics of the audio signals, optimizing the data for further analysis and model training.

#### 3.1.5.1 SAMPLING

Sampling in audio signal processing is the method of converting a continuous audio signal into a digital signal by measuring the signal's amplitude at evenly spaced intervals, or sampling points. This process allows the analog audio wave to be stored and processed in a digital form, making it suitable for applications such as digital playback, editing, and transmission.

The rate at which samples are taken, known as the sampling rate, significantly impacts the quality and accuracy of the digital representation. Higher sampling rates, such as 44.1 kHz (used in audio CDs) and 48 kHz (standard in professional audio), capture more detail from the original signal. The Nyquist Theorem establishes that to accurately reproduce the audio without distortion or loss of information, the sampling rate must be at least twice the highest frequency in the audio signal.

Common sampling rate: 44.1KHz ,48 kHz

Nyquist Theorem:  $f_s \geq 2 f_{max}$

$f_s$  :sampling rate,  $f_{max}$  : maximum frequency component in the signal

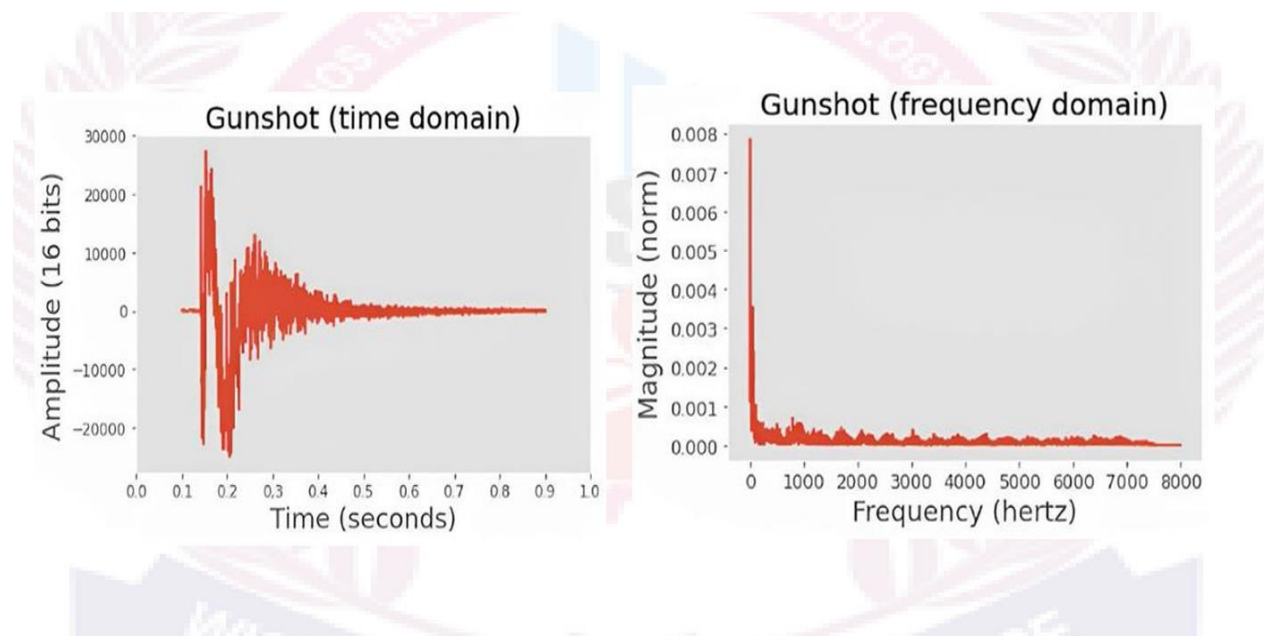
### 3.1.5.2 CONVERTING TIME DOMAIN INTO FREQUENCY DOMAIN

#### THE FAST FOURIER TRANSFORM (FFT)

It is a method used to convert an entire time-domain signal into the frequency domain, providing a global representation of the signal's frequency content. It is ideal for stationary or periodic signals where the frequency characteristics do not change over time. The FFT is efficient in analyzing signals that remain constant, allowing for a detailed understanding of the frequency spectrum. However, it has a limitation in that it cannot provide time-specific information, making it unsuitable for analyzing non-periodic or transient signals.

#### SHORT-TIME FOURIER TRANSFORM (STFT)

It is designed to handle non-stationary signals by dividing the signal into short segments and converting each segment into both time and frequency domains. This method provides a time-frequency representation, which is particularly useful for analyzing signals that change over time, such as speech or music. While the STFT allows for the observation of how frequency components evolve in time, its main limitation is that the time resolution is dependent on the window size used for segmentation, which can compromise the accuracy of frequency det



### **3.1.5.3 MEL SPECTROGRAM REPRESENTATION**

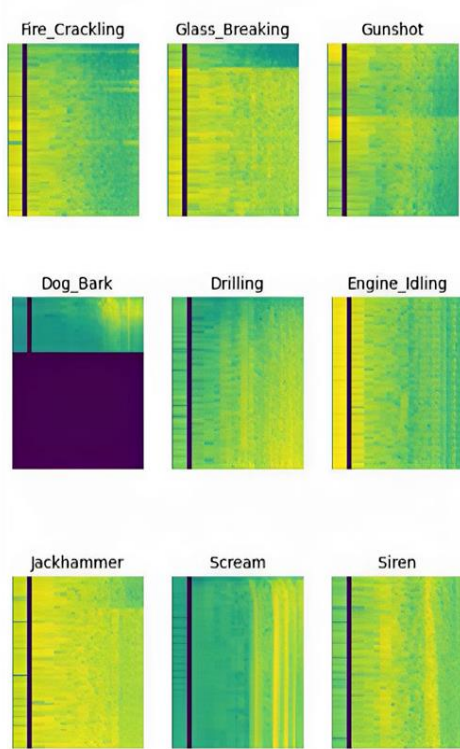
#### **MEL SPECTROGRAM**

A Mel Spectrogram is a time-frequency representation of an audio signal that has been transformed using the Mel scale, which is designed to match the way humans perceive pitch. The Mel scale is nonlinear, compressing higher frequencies and expanding lower frequencies to reflect the human ear's sensitivity to different pitches. However, as the frequency increases, the changes on the Mel scale become less pronounced, with higher frequencies being compressed more. This characteristic allows the Mel spectrogram to reflect how humans perceive sound, giving more importance to lower frequencies and capturing the nuances that are most perceptible to the human ear. To generate a Mel spectrogram, the audio signal is passed through a Mel filterbank, is a series of filters applied to the frequency components of an audio signal to approximate how humans perceive sound frequencies. These filters emphasize lower frequencies by using the Mel scale, creating a compressed representation where each filter output corresponds to a specific frequency band, enhancing the most perceptible aspects of the signal.

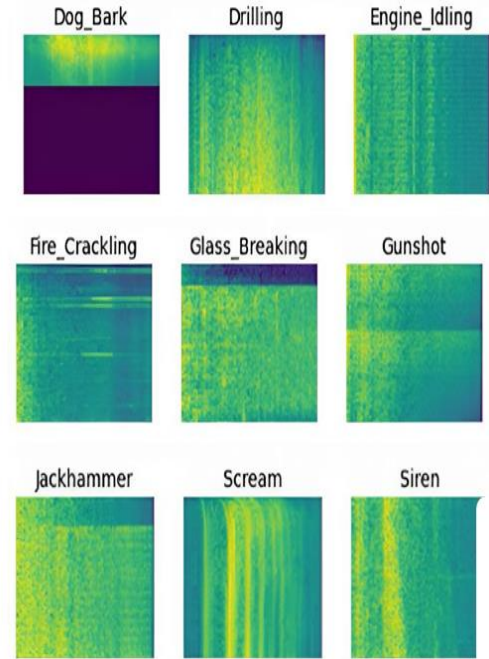
#### **MEL SCALE**

The Mel scale is a perceptual scale of pitch that reflects how humans perceive changes in frequency. Unlike a linear frequency scale, the Mel scale compresses higher frequencies and expands lower frequencies, as the human ear is more sensitive to changes in pitch at lower frequencies than at higher ones. This non-linear scale is designed to model human auditory perception more accurately, making it particularly useful for tasks such as speech processing, where the focus is on features relevant to human hearing. The Mel scale is commonly used in audio signal processing, especially in the creation of Mel spectrograms, which are widely applied in areas like speech recognition and music analysis.





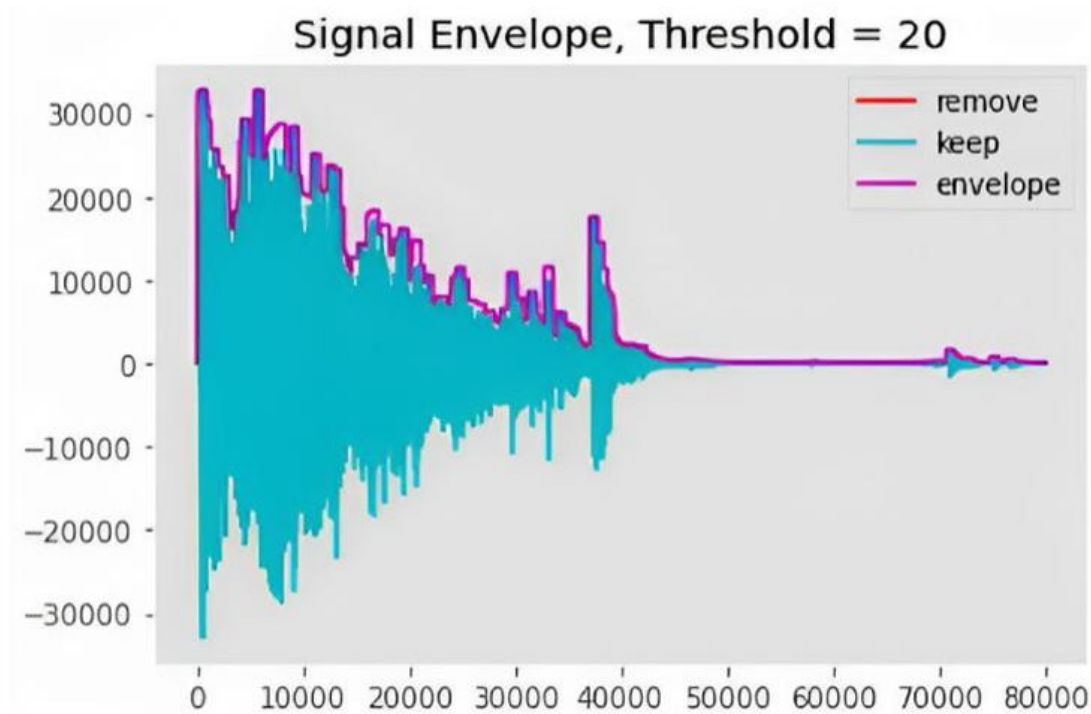
**FIGURE 6.** The decibel spectrogram for one sample audio each from 12 of the 13 classes considered in the research (the same as considered in the time domain plots above).



**FIGURE 7.** The Mel spectrogram for one sample audio each from 12 of the 13 classes considered in the research.

### 3.1.5.4 CLEANING THE SOUND DATA

To prepare audio data for analysis, each file is organized into class-specific folders and converted to the ".wav" format for faster loading, despite its high memory usage. To standardize diverse real-world sounds, audio files are converted to mono and down-sampled to 16,000 Hz, reducing computation time during training and ensuring uniformity across data. Silent regions within the audio, which add minimal value, are removed using a custom signal envelope that filters out low-magnitude sections, retaining only relevant data for analysis. The cleaned audio is split into 1-second intervals, or "delta time" segments, to facilitate efficient model training on manageable samples. This cleaned, segmented data is then stored in a new directory, organized by class, to streamline model training on consistent, high-quality input.



**FIGURE 8.** The signal envelope for one of the audio signals corresponding to “glass breaking”. Here, since the threshold magnitude is considered as 20, any part of the signal below that threshold is not considered, and this can be verified with the help of the non-silent signal envelope created around the signal (visible in purple).

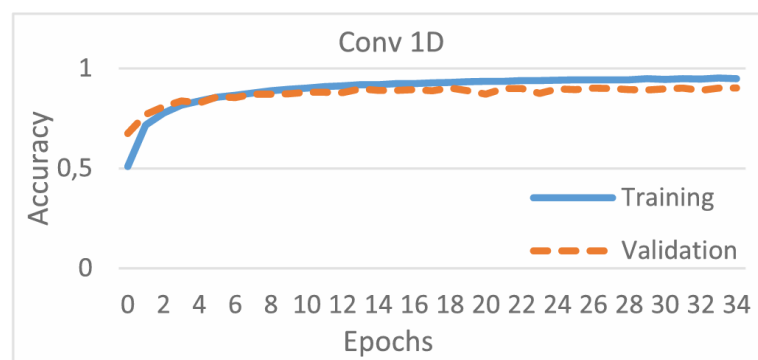
### 3.1.6 APPLIED DEEP LEARNING MODELS

This project explores three deep learning models—LSTM, 1D-CNN, and 2D-CNN—each offering unique advantages for training, analyzing, and classifying audio data. The prediction module is designed to allow selection among these models based on the specific needs of the audio input. A custom data generator prepares and loads cleaned audio data into the models in efficient batches, enabling processing across multiple GPUs for optimal performance during training.

### 3.1.6.1 THE 1D-CNN (1-DIMENSIONAL CONVOLUTIONAL NEURAL NETWORK) MODEL

The 1D-CNN model is designed to capture significant features within fixed-length segments of data, making it effective for time-series data such as audio signals. This model wraps 1D convolutions over time with time-distributed layers, allowing it to process the frequency spectrum of audio signals across time. A permute layer reorders the input data dimensions to align with the time-distributed format, setting time as the primary dimension for the network. The model then applies convolution layers with different activation functions, such as hyperbolic tangent and ReLU, enabling the network to capture nonlinear patterns and avoid issues like vanishing gradients. As it progresses through layers, the network gradually expands its feature space, building complexity until it reaches a dense representation that can effectively feed into classification layers.

Towards the end, a GlobalMaxPooling layer captures the last dimension of features, ensuring efficient use of the model's resources for training. To prevent overfitting, the model incorporates dropout and regularization layers. The final classification is achieved through a dense layer that includes an output SoftMax activation function, converting the outputs into probability vectors for each class, thus enabling categorical classification. By using categorical encoding, the model effectively predicts the class of audio data based on the probability calculated across possible outcomes.

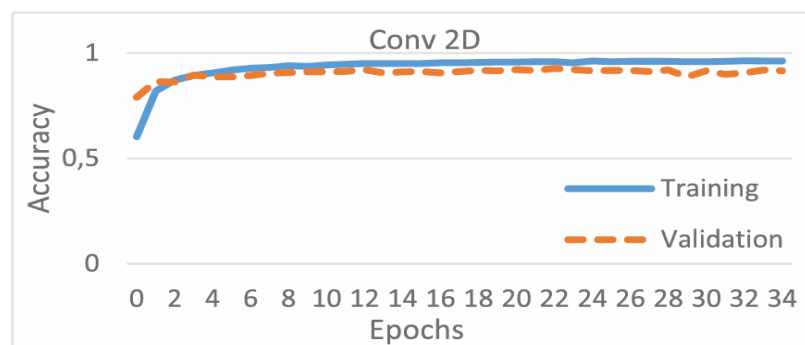


**FIGURE 9.** On running for 35 epochs, the maximum accuracy reached for the 1D-CNN model is found to be 95.2%, with a maximum validation accuracy of 90.2%. The metric used here for considering the best results is the validation loss. The Training (train) vs Validation (test) can be seen in this figure.

### 3.1.6.2 THE 2D-CNN (2-DIMENSIONAL CONVOLUTIONAL NEURAL NETWORK) MODEL

The 2D-CNN model, rooted in the Lenet-5 architecture, is highly effective for analyzing data with spatial dimensions, such as audio spectrograms, due to its ability to capture spatial relationships within the input. This is achieved by using Conv2D layers, where the kernel moves across two dimensions. As the model processes the spectrogram, it recognizes patterns by evaluating adjacent features, which are key in learning the frequency patterns within audio data. The model's architecture consists of successive Conv2D layers, each with progressively more activation units (e.g., 8, 16, 32), and uses ReLU activation functions, except in the first layer, which uses a hyperbolic tangent activation function for improved performance in speech and language-related tasks. MaxPooling layers are added between convolutional layers to downscale feature maps, which helps to retain the most relevant information from each region.

The final layers of the 2D-CNN model closely resemble those in a 1D-CNN, with flattening, dropout, and regularization layers to prevent overfitting. After the last Conv2D layer, the model applies a flattening layer that reduces the output to a one-dimensional array for further processing, avoiding excessive memory usage. This step is particularly crucial because the time intervals set during preprocessing (e.g., 1-second intervals) influence the final dimensions of the input, which affects memory requirements and computational efficiency. The final classifier comprises two dense layers, using 64 activation units in the first dense layer and applying a SoftMax activation function in the last layer, which outputs prediction probabilities for all class.

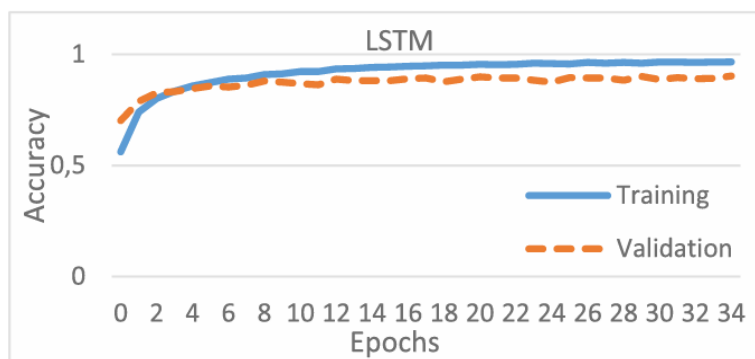


**FIGURE 10.** On running for 35 epochs, the maximum accuracy reached for the 2D-CNN model is found to be 96.3%, with a maximum validation accuracy of 92.7%. The metric used here for considering the best results is the validation loss. The Training (train) vs Validation (test) can be seen in this figure.

### 3.1.6.3 THE LSTM MODEL

LSTM (Long Short-Term Memory) networks, a type of Recurrent Neural Network (RNN), are particularly well-suited for tasks involving sequences of data, such as audio, video, and speech recognition. Unlike traditional feedforward neural networks, LSTMs have feedback connections that allow them to process entire data sequences, capturing long-term dependencies. An essential component of LSTM is the "cell state," which maintains information over time, with gates that regulate the flow of data. These gates, controlled by sigmoid layers, determine which information should be added to or removed from the cell state. The LSTM model in this research is designed to learn features over time by processing input in batches, using a time-distributed dense layer and a Bidirectional LSTM layer to process data in both forward and backward directions for more accurate gradient updates.

The LSTM network progresses by applying a Skip-Connection technique, which concatenates the features from the time-distributed dense layer with those from the Bidirectional LSTM layer. This combined feature set is then processed through additional dense layers with activation units, followed by MaxPooling and Flattening layers. A 1D-MaxPooling layer is used due to the loss of channel information during reshaping. The final classification process involves dropout and regularization layers to minimize overfitting, with two dense layers (one with 32 units and the other with sound classes) used for classification. The output is generated using a SoftMax activation function, which provides a probability distribution for each class, similar to the classification strategies used in the 1D-CNN and 2D-CNN models.



**FIGURE 11.** On running for 35 epochs, the maximum accuracy reached for the LSTM model is found to be 96.6%, with a maximum validation accuracy of 90.3%. The metric used here for considering the best results is the validation loss. The Training (train) vs Validation (test) can be seen in this figure.

### **3.1.7 THE LIVE AUDIO RECORDING AND PREDICTION MODULE**

The prediction module processes real-world audio data by down-sampling the recorded audio to a mono channel with a 16,000 Hz sampling rate and creating a non-silent signal envelope to focus on relevant parts of the signal. The audio is then batched into 1-second intervals, and predictions are made using the argmax function on hot-encoded probabilities, averaging them over the entire sample for final classification. If the audio is identified as related to a potentially hostile environment, such as fire crackling, glass breaking, gunshots, or screams, an automatic alert is sent via email, SMS, and WhatsApp to registered contacts. The email includes the recorded audio file, allowing recipients to listen to the event that triggered the alert, with the recording saved and processed by the audio recording module.

The audio recording module captures live audio from the environment and stops recording when no noise is detected. It then performs several pre-processing steps, including trimming silence from both ends of the recording, normalizing the volume across the entire recording, and adding a 0.5-second audio padding at both ends to ensure no crucial data is lost during playback. Once pre-processing is complete, the audio is saved as a ".wav" file at a specified location. The prediction module is then automatically triggered to analyze the recorded audio using one of the three deep learning models defined by the user, performing all necessary steps to identify the class of the audio.

## CHAPTER 4

### EVALUATION

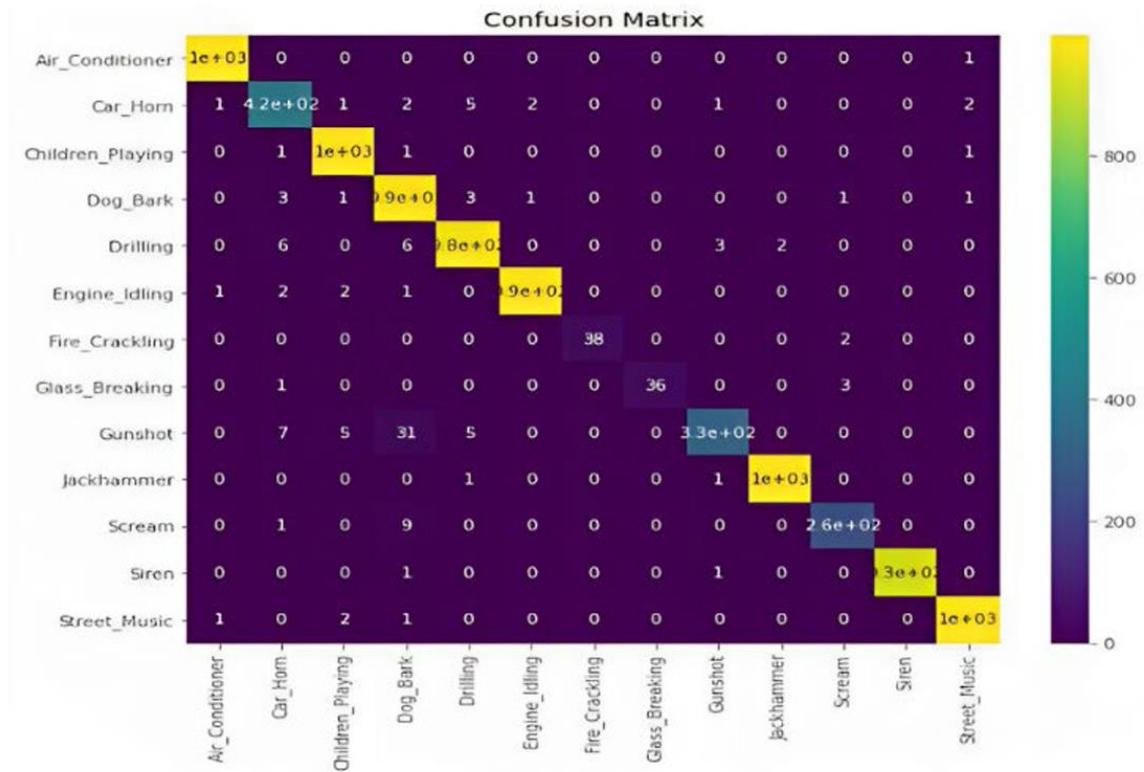
This research aimed to identify real-world audio data using deep learning-based multi-class classification and determine if the identified sound type indicates a potentially dangerous or threatening environment. The focus was to create a threat alert system that automatically sends an alert via email, SMS, and WhatsApp, including an audio recording of the environment. The system successfully records, preprocesses audio, runs the prediction model, and issues an immediate alert when a threat is detected.

#### 4.1 PERFORMANCE OF DEEP LEARNING MODELS

##### 4.1.1 CONFUSION MATRIX

The confusion matrix is a crucial tool in the performance analysis of deep learning models, providing a detailed breakdown of how well the model performs in classifying data. It displays the true positives, true negatives, false positives, and false negatives for each class, helping to identify where the model makes correct predictions and where it misclassifies data. In the context of multi-class classification tasks, the confusion matrix allows for a clearer understanding of the model's accuracy by showing which classes are more likely to be confused with each other. This information is vital for evaluating model performance beyond just accuracy, as it highlights specific areas where improvements, such as better feature extraction or model tuning, may be needed. It includes four key components: True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN), which indicate the model's correct and incorrect predictions. For binary classification, the matrix is a 2x2 table, with rows representing actual classes and columns representing predicted classes. These components are essential for calculating metrics like accuracy, precision, recall, and F1-score to assess the model's performance.





**FIGURE 12.** The confusion matrix attained in live event prediction.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1-score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$



## CHAPTER 5

### CONCLUSION

This research presents a novel software-based threat identification system that enhances personal safety by analyzing ambient noise to detect potentially dangerous situations in real time. Unlike traditional security measures that rely on manual intervention or hardware-based monitoring, this system leverages deep learning models—1D-CNN, 2D-CNN, and LSTM—to classify environmental sounds and determine whether a person is in distress. By continuously recording live audio, the system processes and predicts the nature of the sound, achieving high accuracy levels of 95.2% with 1D-CNN, 96.3% with 2D-CNN, and 96.6% with LSTM, culminating in an average accuracy of 96.03%. The ability to select between these models allows users to optimize the system's performance based on the complexity of the live audio input, making it adaptable to a wide range of real-world scenarios. Upon detecting a potential threat, the system automatically alerts emergency contacts via WhatsApp, SMS, or email, ensuring rapid response and intervention when necessary.

Beyond immediate safety applications, this research demonstrates the broader potential of deep learning and NLP techniques in real-time event detection. By integrating advanced machine learning with audio signal processing, the system sets a foundation for future enhancements, including expanding the dataset to encompass a more diverse range of sounds, improving model precision, and incorporating multi-modal data sources such as video or biometric signals. Additionally, refining the models to recognize complex acoustic patterns in noisy environments could further enhance detection accuracy. This work highlights the transformative role of AI in security applications, offering a scalable, automated approach to threat detection that can be deployed in personal safety devices, smart surveillance systems, and law enforcement operations. Ultimately, the research successfully addresses a critical challenge in security technology, demonstrating how deep learning can be harnessed to analyze ambient sounds and provide timely alerts in potentially life-threatening situations.

## REFERENCES

- [1] T. P. Suma and G. Rekha, "Study on IoT based women safety devices with screaming detection and video capturing," *Int. J. Eng. Appl. Sci. Technol.*, vol. 6, no. 7, pp. 257–262, 2021.
- [2] P. Zinemanas, M. Rocamora, M. Miron, F. Font, and X. Serra, "An interpretable deep learning model for automatic sound classification," *Electronics*, vol. 10, no. 7, p. 850, Apr. 2021.
- [3] D. G. Monisha, M. Monisha, G. Pavithra, and R. Subhashini, "Women safety device and application FEMME," *Indian J. Sci. Technol.*, vol. 9, no. 10, pp. 1–6, Mar. 2016.
- [4] G. Ciaburro and G. Iannace, "Improving smart cities safety using sound events detection based on deep neural network algorithms," *Informatics*, vol. 7, no. 3, p. 23, Jul. 2020.
- [5] J. Cao, M. Cao, J. Wang, C. Yin, D. Wang, and P.-P. Vidal, "Urban noise recognition with convolutional neural network," *Multimedia Tools Appl.*, vol. 78, no. 20, pp. 29021–29041, Oct. 2019.

Received December 13, 2019, accepted January 5, 2020, date of publication January 13, 2020, date of current version January 22, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.2965947

# Object Removal Software Test Automation

DEBDEEP BANERJEE, (Member, IEEE), KEVIN YU, AND GARIMA AGGARWAL

Qualcomm Technologies Inc., San Diego, CA 92121, USA

Corresponding author: Debdeep Banerjee (debdeepb@qti.qualcomm.com)

**ABSTRACT** This paper considers the problem of designing a test method and robust test automation for evaluating the functionality of computer vision algorithms for object removal and image reconstruction of the removed area. An object removal algorithm is used to select and remove a region in an image. After the removal of the object, the object removal algorithm regenerates the area in the image from which the object was removed, similar to photoshop. We developed an algorithm in MATLAB that accesses the features of the macroblocks from the region that adjoins the removed object and correlates these macroblocks with the actual area that is regenerated. The SSIM (structural similarity index) score is calculated by comparing the ground truth with the image generated from image reconstruction after removal of the object. Finally, we calculate the artifact area present in the regenerated region to determine the algorithm quality. To validate this method, the authors compare a 3<sup>rd</sup>-party object removal solution with an in-house developed solution using the proposed approach. This comparison shows that the SSIM score is 1.55% higher than the 3<sup>rd</sup>-party solution and 26.3% lower in artifact areas. Besides the results, this paper also describes in detail the procedure to automate the use case of object removal test automation to scale up the test coverage.

**INDEX TERMS** Software engineering, software testing, image processing, image restoration, multimedia.

## I. INTRODUCTION

This paper discusses a validation strategy for testing an object removal algorithm. We designed test-automation-based algorithms that evaluate the area that is regenerated after an object is removed. The object removal software test automation is developed to validate the functionality, performance, and stability of the object removal algorithm. The test automation uses image quality algorithms such as the SSIM (structural similarity index) to evaluate the comparison of the image that was reconstructed after the object was removed with the ground truth. We also use algorithms to detect the presence of artifacts in the image reconstructed area.

We compare the luma values of the macroblocks of the region that adjoins that from which the object was removed. Then, we apply the k-means clustering algorithms to confirm the number of color clusters in the images and compare this number with the ground truth to evaluate the functional verification of the image reconstruction.

There has been research in the field of denoising and artifact removal of an image using Weiner filter etc., [1]. The effect of object removal from an image has to be filled in a

visually plausible way [2]. We develop a test automation that measures the quality of the regenerated area using

The associate editor coordinating the review of this manuscript and MATLAB software after the object removal algorithms are used. The challenge for the validation team is to programmatically determine the quality of the removed objects from the images. The object removal algorithm needs to regenerate the removed area so that it blends with the surrounding pixels of the object. We identify an approach to effectively generate test images and ground truth images. The output image results can be compared to the ground truth image using the SSIM (structure similarity index) measurement to quantify the regenerated area quality. Finally, a computation is performed using machine learning algorithms such as k-means clustering to detect artifacts and obtain the artifact area percentage. We deterministically assign a result to these test cases. Saliency maps are applied for determining objects and applying masks (green in color) to them based on the object detection.

The object removal applications require us to select a region in the object to remove. If we apply a mask to the object that needs to be removed, we can apply swipe events for the area selected in the mask through touch event-based

automation. The challenge is to detect and select the object and then color the object for the mask. We apply saliency maps for object detection using MATLAB, and we color the region of the saliency map for the mask generation.

The results of a comparison with other Android applications approving it for publication was Habib Ullah.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see <http://creativecommons.org/licenses/by/4.0/> VOLUME 8, 2020 12967

performed with different test vectors, including a cluttered background, a non-cluttered background, etc. We perform competitive analysis tests with a 3<sup>rd</sup>-party software application using the test automation on different types of background test images, quantitatively proving that our object removal application regenerates background areas that have

better image quality. We compare the object removal latencies for the device that performs with hardware processors and that uses hardware acceleration for performing the object removal algorithms. For the results, the comparative Android application does not use hardware acceleration on the hardware processors. We observe noticeable gains in the latency measurements for the object removal algorithms with hardware acceleration.

This test automation strategy is evaluated for inclusion at various software integration points so that we can detect any regressions in the software code churns. If a regression is detected, the failures are triaged, and if a failure is diagnosed as a software issue, it is debugged and fixed by the software development team. This approach helped us maintain the software quality and detect issues early in the software development lifecycle. In particular, observing problems late in the software development process can lead to significantly higher costs of solving the problem.

## II. MOTIVATION

The goal of developing the test automation strategy was to measure the accuracy and performance of object removal algorithms qualitatively. The development of a robust automation helps us consistently benchmark the functionality and performance test results.

The test automation strategy enables us to apply machine learning algorithms such as k-means clustering to perform color segmentation of the image. The test automation strategy performs feature extraction of the adjoining macroblocks from which the object was removed and compares them to the regenerated area. Depending on the correlation values, the test automation strategy can objectively assess a pass or fail result for the test case. The test automation strategy also uses correlations to compare the similarity of the features of the adjoining macroblocks to the regenerated area. Any unusual colors that are formed during the regeneration of the removed object can lead to the creation of additional color segments and decrease the correlation values. Therefore, the test automation strategy

can detect abnormal color patches, which are distortions in the regenerated area from which the object has been removed.

Stability tests are crucial to software testing but executing the same test cases multiple times can be tedious. Thus, performing object removal are obtained, and testing is

writing and deploying a reliable test automation strategy that can perform functional validation of the test cases can be beneficial. In addition, many issues, such as memory leaks, are detected after several iterations of testing. These stability issues may not be detected during functional testing. The test automaton collects logs for each iteration, which helps the engineers effectively triage the failures based on log analysis and the captured images.

We intend to reduce the workload of manual testing. Manual testing can be a tedious and error-prone process. Thus, if the automation is reliable, test engineers can focus on new test software development efforts, writing new test plans and enhancing test coverage by carefully evaluating the software architecture of the algorithms and developing new test automation strategies for testing them. The test automation strategy can be deployed at several points of the software integration cycle. Testing code churn early in the software development process can lead to the detection of regression issues. The test case failures are triaged and then software failures are debugged and fixed by the software developers. Using this approach, we can efficiently detect the issues early in the process.

## III. BACKGROUND AND RELATED WORK

The research field of artifact removal of an image involves development of image reconstruction techniques. Image reconstruction requires image smoothing techniques. The image smoothing algorithms use techniques for reconstructing the smoothly changed shape and intensity of the original image [3]. Research has been done to apply deep network architecture for removing rain streaks from an image. This has helped in improved rain removal from images and much faster performance in object removal techniques [4]. The image reconstruction techniques are discussed after either part of the image is corrupted [5].

A clustered differential pulse code modulation method with removal of local spectral outliers has been studied and analyzed for lossless compression of images [6]. Image processing algorithms like 3D reconstruction and hand jitter reductions have been implemented on mobile phones. Validation techniques involve design and development of test automation for validating these image processing algorithms [7], [8]. Techniques have been developed to detect the types of noise artifacts for removal in experimental data and analysis [9]. Test automation development for embedded software-based image rectification techniques has been done

for validating the automatic color, sharpness of images [10]. Face recognition software has been popular for image processing in mobile phone. Validation techniques for this feature testing of face recognition has been discussed with performance metrics [11].

Test automation exists for automation image processing systems of high precision X Ray and Neutron imaging applications [12]. Research has been done for development of testing methods for designing artificial scenes and automatically generating virtual images with precise annotation [13]. Test systems can calculate the accuracy of image processing capture systems like fisheye cameras [14]. The accuracy calculations are crucial for any test system to analyze the performance of the system and the software. In this paper we discuss the test automation strategies for the object removal algorithm.

TABLE 1. Snapshot of test cases in the object removal test plan.

| Test Name                                                      | Background      | Object to Remove |
|----------------------------------------------------------------|-----------------|------------------|
| Remove multi-colored object from uniform background            | White wall      | Mannequin head   |
| Remove uniform simple shaped object from textured background   | Textured fabric | Sticky note      |
| Remove complex colored object from repeated pattern background | Brick wall      | Wall Poster      |
| Remove complex shaped object from complex background           | Wall poster     | Building model   |

#### IV. SOLUTION

##### A. SOFTWARE OVERVIEW

Object removal is a digital image editing solution. By editing an image, the user can remove the unwanted content from the image. The libraries provide an application programming interface (API) in which the masked region is an input parameter, providing flexibility to the user to select the content of his/her choice. The object removal feature is preferred when obtaining more personalized images.

##### B. OBJECTIVES

The goal of the object removal software is to remove the content in the selected regions and fill-in the holes of these regions in a visually plausible way. An end-to-end test application automation framework is required to test these features for a large data set and quantitatively evaluate the object removal quality.

In our object removal test plan, there are test cases that involve with combination of different types of background and objects to remove. The backgrounds can be categorized into uniform colored simple background, textured background, repeated pattern background, and complex color/textured background. The objects to be removed from

the backgrounds are uniform and multi-colored objects as well as simple and complex shaped objects. The combination of these objects and backgrounds form a good representation of scenarios that user will be using the object removal app for. Table 1 shows a snapshot of sample test cases in our object removal test plan.

Since we have hundreds of these test cases to fully evaluate the object removal software's performance, manually testing them is time consuming and object removal quality result is subjective. An automated process is needed for object removal software testing. The main challenges of automating this process are the remove object selection, ground truth/test image generation and object removal/background restoration quality postprocessing. Our objectives are to find automated solutions to these pain points and to establish a complete automated workflow.

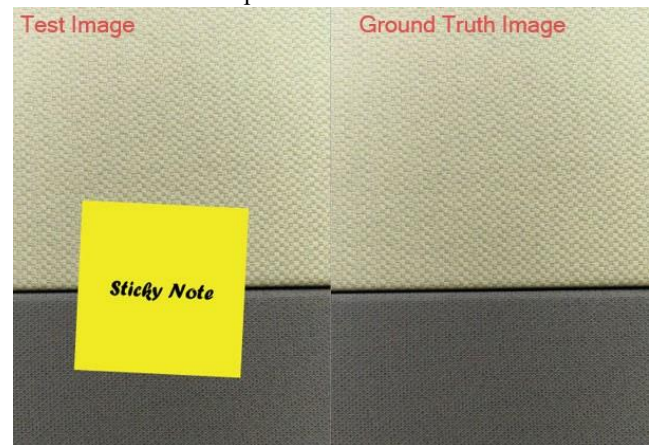


FIGURE 1. Regenerated test image and ground truth image pair.

##### C. GROUND TRUTH IMAGE AND TEST IMAGE GENERATION

To effectively evaluate the object removal and background restoration quality, a perfect background is needed as the ground truth before applying object removal. And we have come up with two methods to generate ground truth image and test image.

One method is to select available images that are only composed of background that are either procured from gallery or image database and use them as ground truth images. Then, we apply image overlap using image cloning software such as Photoshop, and subsequently add object images on top of the background image. The test images are generated composed with the object and the background, as shown in figure 1.

The advantage of this method is the ground truth image is the same as the background in the test image. The image comparison result in the post-processing stage will be highly accurate. But the problems for this method are the object image has to be edited using image software cut to its own



shape before overlap to the background, and also the composed image does not fully represent the natural scene taken with both foreground object and background in the same picture.

The other method of generating ground truth and test images is by taking snapshot of the actual background and object placed in front of the background using camera. When large number of real-world test data images are needed, a systematic way of generating them is necessary. We have utilized the latest technology of robotic arm system to help capture these ground truth and test images.

We have a 6-dof motion robotic arm setup in a closed-space test lab, where we have full control of light intensity and background setup, as shown in figure 2. Different backgrounds can be setup on multiple regions of the lab wall. They range from single color to multiple colors, single texture to complex texture, representing the background requirements of the test cases. The objects from single to complex shapes in the test cases requirements are placed in front covering part of



FIGURE 2. Robotic arm pointing the camera at different test scenes.



FIGURE 3. Ground truth and test images captured using the robotic arm.

the backgrounds. The combination of the objects and the backgrounds create the scenes for the ground truth and test images. The challenge is to capture these images properly so that the background in the ground truth image appears the same in the composition test image for accurate comparison.

Using a high precision robotic arm, there will be only  $\pm 0.03$  mm position error for every meter the robot moves. This means the robotic arm is able to hold the camera device and move it to the accurate position to take snapshots every time. A 6-dof robotic arm has 6 joints, by rotating these joints using controller the tip of the robotic arm can be extended to the desired position to take snapshots. The robot position can be stored to the controller and use robot programming software like WINCAPS3 or ORiN2 to automate the robot motions moving through each saved position in front of different backgrounds. Using robotic arm for ground truth and test image capture proves to be much more accurate than manual capture by analyzing the comparison results. And it also saves valuable time in this test data generating process. On the camera side, the settings like focus and exposure have to be set off from automatic mode. This is needed to prevent exposure and sharpness difference between the ground truth and the test images. The captured results are shown in figure 3.

Generating ground truth and test images using image capture method has its advantages. The generated images have more realistic compositions like the shadows and light exposures. Also, when the required background and test images are not available, we can use this method to create the scenes and generate the test data. The downside for image capture method is there will always be a delta between the ground truth image background and the test image background. However, this delta can be minimized by using the robotic arm for image capture and have the right camera settings. Depending on the test case requirements, the image capture method and the image overlap method can both be effective in generating ground truth and test images.

#### D. GENERATION OF THE GREEN MASK BY OBJECT DETECTION

The next test automation challenge is the removal of the region selection. Using fingers or a pen to highlight the removal region can be difficult on complex shapes. In addition, this approach has a large capacity for human error over repeated attempts. We identify an approach to select the object to be removed in an image by highlighting it in a green mask. We apply saliency maps [15]–[17] to detect objects in the images. The saliency maps are generated based on the features of the images, such as color, intensity, etc. The saliency maps are normalized and overlaid on the original image for object detection.

The saliency maps can be created based on the graph-based saliency algorithm. If we are given an image  $I$  the most significant location of the images can be computed. The feature maps based on color, intensity etc are computed and then these feature maps are normalized for generate the saliency map.  $M(i, j)$  corresponds to locations in the image  $I$  which are unusual to its neighborhood and can be related to higher values of the activation map.

To calculate the dissimilarity between the maps of  $M(i, j)$  and  $M(p, q)$  the following formula is used.

$$d((i, j), (p, q)) = \left| \log \frac{M(i, j)}{M(p, q)} \right|$$

Once the saliency map is generated a weight will be associated to the directed edge from node  $(i, j)$  to node  $(p, q)$ .

$$w_1((i, j), (p, q)) = d((i, j), (p, q)) \cdot F(i - p, j - q),$$

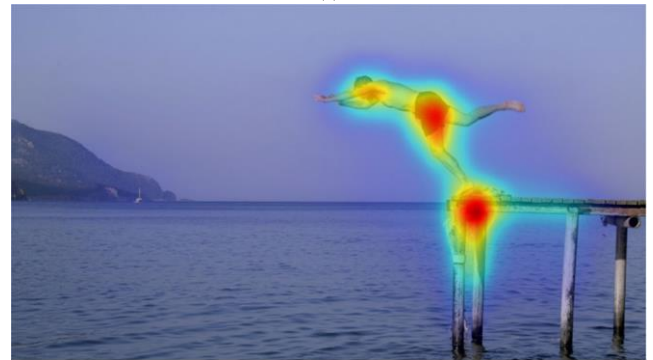
$$\text{where } F(a, b) = \exp\left(-\frac{a^2 + b^2}{2\sigma^2}\right).$$

Here sigma is a free parameter of our algorithm. Once the feature maps are formed comprising of individual features like color, intensity etc the feature maps are normalized. Winner Takes All Computation to detect the point of highest saliency in the map at any given time. This information is used for compute object detection and recognition.

Once the saliency maps are generated and the objects are detected, we use the map area to color the object and generate the mask. Please refer to figure 4a, 4b and 4c for details of the saliency map generation and the generation of the masks in the image.



(a)



(b)



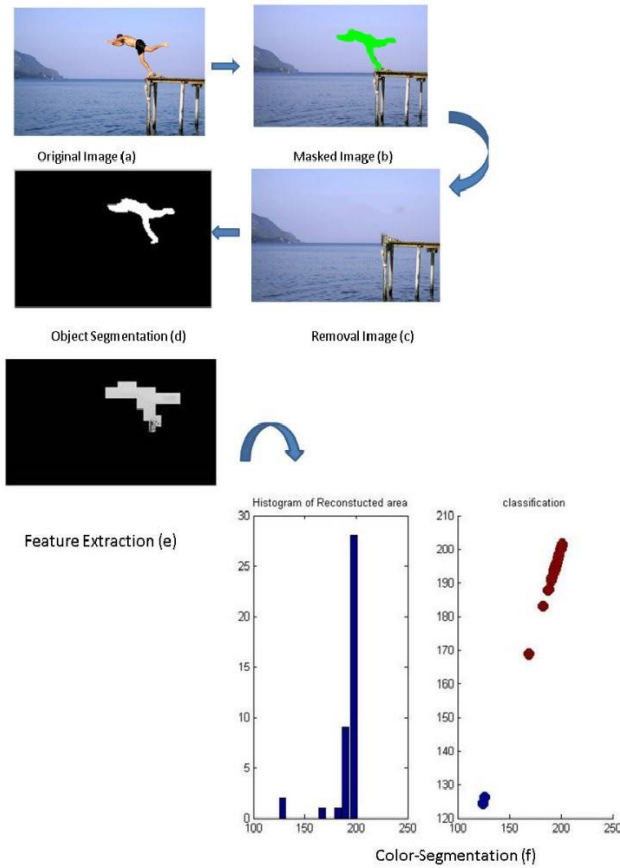
(c)

**FIGURE 4.** (a) The original image from which the object (man diving into the water) needs to be removed. (b) The saliency maps are generated and overlaid on the image. Please note that the saliency maps are used for object detection. We use MATLAB code for the saliency map generation. (c) The saliency maps are generated, and the area of the saliency maps is used for coloring the area and generating the mask.

We then use these masks in our test automation. The green mask is selected using touch events, and the object removal algorithms are applied to remove the area in the mask. The object removal algorithm then regenerated the removed area. The algorithm analyzes the features, such as color and luma values of the adjoining pixels of the removed area, and then regenerates the area.

#### E. OBJECT REMOVAL AND BACKGROUND RESTORATION POSTPROCESSING ALGORITHM

In this paper, we propose a postprocessing algorithm for analyzing the quality of a reconstructed region. The algorithm



**FIGURE 5.** Stepwise presentation of the algorithm: (a) Original image with the object; (b) Masked object with green color; (c) Resultant image after using the object removal feature; (d) Object segmentation from the masked image; (e) Feature extraction from the removal image; (f) Color segmentation using the luminance value.

is designed to segment the object from the image and find the neighboring pixel blocks using the standard deviation. Then, the luminance value is calculated for feature extraction. Finally, a computation is performed using SSIM (structural similarity index) and k-means clustering to obtain the removal artifact percentage.

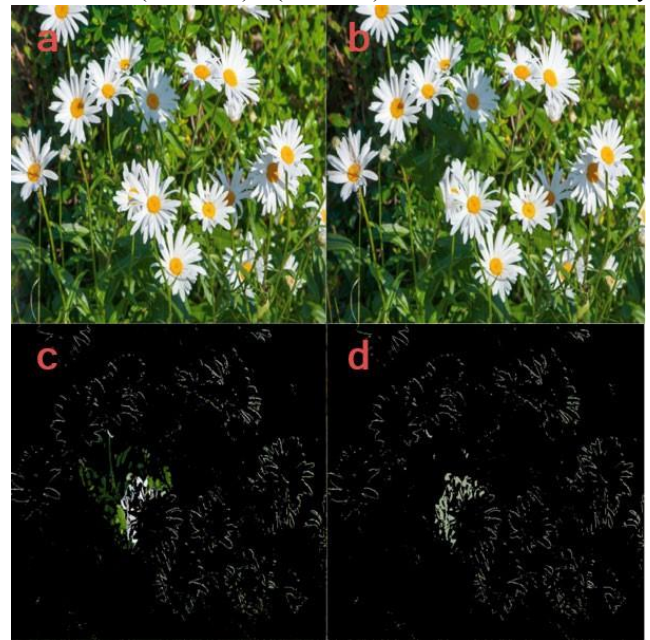
### 1) OBJECT SEGMENTATION

Object segmentation is required to identify the erased object in the image. The object is defined by using a predefined mask with  $RGB = (0, 255, 0)$  values. In this manner, the object is segmented in the form of a binary image, which is denoted as  $I(x, y)$  and satisfied  $I(x, y) = I(x, y, 1) == 0, I(x, y, 2) == 1, I(x, y, 3) == 0$ .

### 2) FEATURE EXTRACTION

The information of the neighboring pixels is needed to extract the information of the filled area in the output image.

The neighboring pixels of the object are identified by splitting the binary image into overlapping blocks of  $b \times b (32 \times 32)$  pixels. In this way, the whole image of size  $M \times N$  is divided into  $(M - b + 1) \times (N - b + 1)$  macroblocks. To identify



**FIGURE 6.** Object removal/reconstruction artifact extraction: (a) Ground truth background image; (b) Reconstructed background image after object removal; (c) Color segmentation comparison between the output and ground truth images; (d) Single color artifact extraction.

the blocks of interest, the standard deviation is calculated for each macroblock. The maximum deviation is observed in the blocks that contain the object pixels and neighboring pixels. For feature extraction, the output image is also divided into overlapping blocks of  $b \times b (32 \times 32)$  pixels. The blocks that had maximum deviation and contained the removal object are selected for feature selection for further processing. Feature selection is performed by calculating the luminance value ( $Y = 0.299R + 0.587G + .114B$ ) of each selected block. Luminance quantifies the trichromatic intensity perception of humans [4].

### 3) COMPUTATION

After identifying the blocks that contain the reconstructed region and their neighbors in the output image, a computation is performed using SSIM (structural similarity index) between the output image and the ground truth image on the reconstructed region. A high SSIM score (close to 1) means the object removal has performed well, and the reconstructed

background that is generated is almost identical to the original ground truth image. For artifact detection and calculation on the output image, color segmentation is performed on the output image and compared to the ground truth image using the k-means clustering algorithm. This approach highlights the areas where there are color differences between the two



images. Then, the delta regions are extracted into a single color, highlighting the artifacts. This process is demonstrated in figure 6.

#### F. TEST AUTOMATION WORKFLOW

With the solutions to all three pain points, a complete test automation workflow can be established. Before running

TABLE 2. Post processed image result scores comparison table.

| Types of background          | Number of images tested | SSIM Score         |                            | Artifact Percentage |                            |
|------------------------------|-------------------------|--------------------|----------------------------|---------------------|----------------------------|
|                              |                         | Object removal app | 3 <sup>rd</sup> -party app | Object removal app  | 3 <sup>rd</sup> -party app |
| Uniform background           | 45                      | 0.971              | 0.968                      | 0.37%               | 0.40%                      |
| Textured background          | 62                      | 0.872              | 0.839                      | 4.39%               | 6.64%                      |
| Repeated background          | 38                      | 0.684              | 0.679                      | 11.94%              | 13.23%                     |
| Complex/cluttered background | 55                      | 0.843              | 0.839                      | 3.94%               | 6.71%                      |

the automation, background images are set as ground truth images. The background images are then added with objects layers on top. Using object detection, these objects are predefined with green masks as the input images. When the object removal test automation is executed on the input image, a Perl script is written to open the Android test application, to select the predefined mask using touch events, to perform the removal, and to save the edited image. The postprocessing algorithm evaluates the quality of the regenerated area based on the features of the macroblocks that are near the object that is being removed and then applies the SSIM (structural similarity index) scores. The postprocessing algorithm also applies the k-means clustering algorithm to the image to perform color clustering to identify artifacts. A computation is performed using k-means clustering to obtain the artifact percentage. Please refer to figure 7 for details.

## V. RESULTS

### A. OBJECT REMOVAL PERFORMANCE RESULTS ON IMAGE QUALITY

In our test automation, we test 200 test vectors, including the dataset images from Caltech101 [18]. The images are categorized with different backgrounds such as uniform, textured, repeated and cluttered. These different types of backgrounds represent different challenges for the background restoration algorithms. To compare the performance with other object removal apps on the market, we perform the end-to-end automated test on both the object removal app and the 3rd-party app. The postprocessed test scores are averaged and are shown in table 2.

From this table, we can see that the object removal app and the 3<sup>rd</sup>-party app performed similarly on the same type of backgrounds, while the object removal app received slightly better SSIM scores (approximately 1.55%) on all four types of backgrounds. The SSIM scores comparison data are plotted as bar graphs in figure 8.

The SSIM score measures how similar the comparison images are; in this case, the images are the output image and the ground truth image. These data show that the SSIM

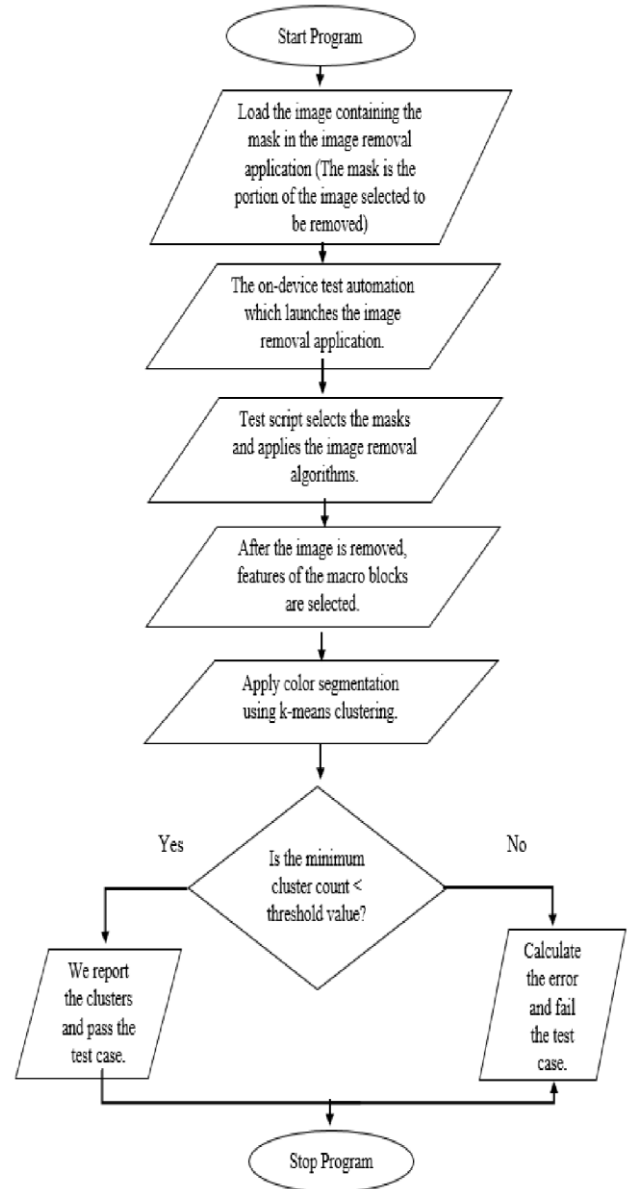


FIGURE 7. Flowchart of the end-to-end object removal test automation.

scores vary between different types of backgrounds. For uniform colored backgrounds, it is easier to reconstruct the background of the removed region. Therefore, this category received the highest SSIM score. On the repeated background images, the background contains even or uneven

repeated patterns. The background reconstruction algorithm must analyze these patterns and fill the removed space with the proper patterns. This task proves to be quite challenging, and this category received the lowest SSIM score. As an example, figure 9a shows the image of a brick wall before the removal of the blue poster region and the ground truth image of the brick wall without the poster. In figure 9b, the left side is the object removal app output of the reconstructed brick wall image after poster removal, and the right side is the 3<sup>rd</sup>-party output image. It is clear that both software solutions were

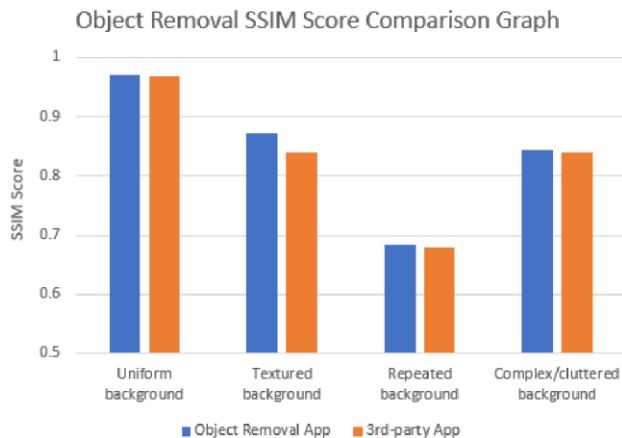


FIGURE 8. Comparison of object removal SSIM scores between the object removal app and the 3<sup>rd</sup>-party app.

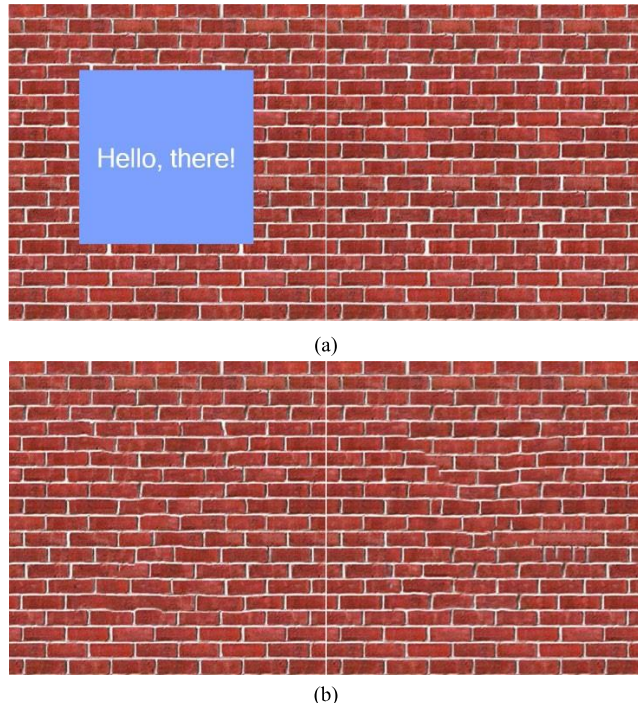


FIGURE 9. (a) Image of a brick wall with a blue poster and ground truth image without the poster. (b) Object removal output images (left one from the object removal app and right one from the 3<sup>rd</sup>-party app).

unable to reconstruct the wall pattern perfectly compared to the ground truth. However, the left image's reconstructed pattern more closely resembles the actual pattern.

Another example of the textured background object removal output results is shown in figure 10. Both object removal software solutions remove the yellow sticky note on the image from figure 1.

Both output images are able to reconstruct the removed sticky note area with the proper surrounding texture. However, both images have an artifact of a skewed line on the space separating the two textures. We also occasionally observe that when selecting the removal regions with the 3<sup>rd</sup>-party app, a diagonal line is applied to the image

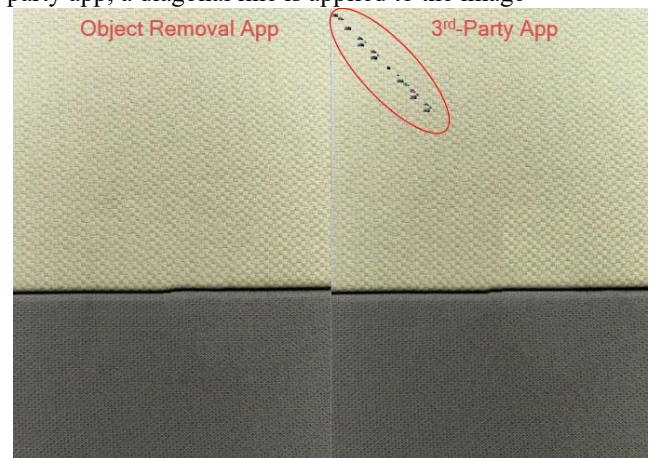


FIGURE 10. Textured background object removal output images from the object removal app and the 3<sup>rd</sup>-party app.

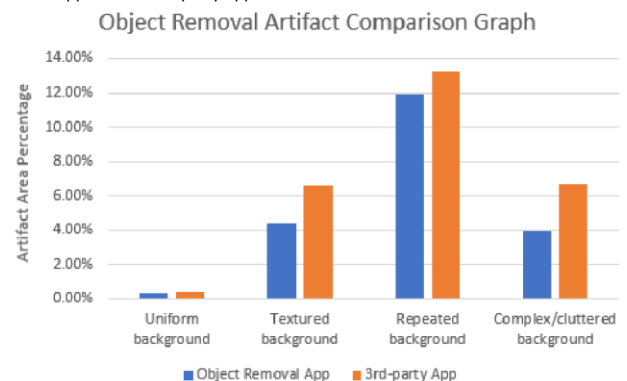


FIGURE 11. Object removal test output artifact area percentage comparison graph.

along with the removal region. This line creates an additional artifact on the output result image. This issue emphasizes the importance of artifact detection and measurement in the object removal test automation.

The artifact percentage used in our test automation is calculated by dividing the areas of the processed output image highlighted with the artifact by the area of the removed (masked) regions. The artifact percentage data from table 1 is converted into a bar graph in figure 11. This graph shows that the object removal app generally has smaller artifact

regions (approximately 26.3%) than the 3<sup>rd</sup>-party app. The uniform background category shows almost zero artifacts. Textured and complex/cluttered backgrounds have a moderate region of artifacts. In addition, repeated backgrounds have the largest region of artifacts, as expected. This artifact result is aligned with the SSIM score result.

Figure 12 shows an example of the processed output images for artifact detection. These images are created by removing a butterfly from the flowers previously used in figure 6. The left image in figure 12 is processed from the object removal app, and the right image is processed from the 3<sup>rd</sup>-party app. The highlighted regions have artifacts. The image on the right clearly shows a larger highlighted area.



FIGURE 12. Processed artifact areas from the object removal app output and the 3<sup>rd</sup>-party app output.

TABLE 3. Object removal latency result comparisons.

| Types of background | Image resolution | Object Removal App latency (sec) | 3 <sup>rd</sup> -Party App latency (sec) |
|---------------------|------------------|----------------------------------|------------------------------------------|
| Uniform             | 1280x720         | 0.16                             | 0.90                                     |
| Textured            | 1280x720         | 0.53                             | 1.10                                     |
| Repeated            | 1280x720         | 0.39                             | 0.96                                     |
| Cluttered           | 1280x720         | 0.84                             | 1.41                                     |

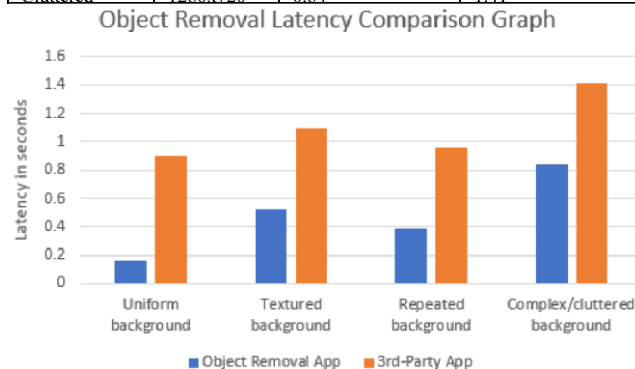


FIGURE 13. Latency comparison graph on object removal process.

From running the automated test on both object removal software apps with the 200-test image dataset, we can conclude the object removal app performs better than the 3<sup>rd</sup>-party app in object removal quality. In addition, we find that

the algorithm was most accurate for images with the following characteristics:

- 1) Constant color background. (e.g., clear sky)
- 2) Small textures.
- 3) Sharp edges.
- 4) Small mask sizes.

#### B. OBJECT REMOVAL PERFORMANCE RESULTS ON LATENCY

During the object removal test, we also measure the latency of the object removal process on the hardware accelerated object removal app and the 3<sup>rd</sup>-party Android app. The measurement is taken from Android logs, and it is measured across all four categories of backgrounds. The latency results are averaged and are presented in table 3.

The latency comparison data are converted to bar graphs shown in figure 13 for a better visual comparison.

From the results, we concluded that the latency for the object removal is approximately 56% lower in the case of the object removal app that use hardware acceleration than that of the 3<sup>rd</sup>-party Android app. The hardware acceleration is achieved by using signal processors, which executes the object removal algorithms. The complexity of the object removal algorithms increases with cluttered backgrounds, and we observe that the latency increases as we test from uniform backgrounds to cluttered backgrounds.

## VI. CONCLUSION

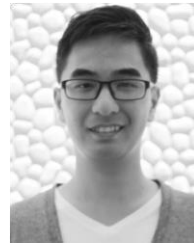
The object removal test method and automation strategy provide an option for executing algorithm tests for each software release and benchmarking the results by the software release. New test contents can be added to the test automation framework to expand the test coverage for more specific scenario object removal algorithm testing.

The proposed test method and automation strategy can be used for executing functional, performance, stability and adversarial test cases. They can also be scaled to multiple software products and produces consistent and reliable test results. We are constantly reviewing the test plan and adding new test cases according to the new features and optimizations that are added by the software development teams based on the object removal feature. The goal is to scale up the test automation so that test engineers can utilize their time for the development of new feature tests and to ensure that this test automation strategy can execute regression test cases reliably.

## REFERENCES

- [1] A. Kazakeviciute, C. J. H. Ho, and M. Olivo, "Multispectral photoacoustic imaging artifact removal and denoising using time series modelbased spectral noise estimation," *IEEE Trans. Med. Imag.*, vol. 35, no. 9, pp. 2151–2163, Sep. 2016.
- [2] A. Criminisi, P. Perez, and K. Toyama, "Region filling and object removal by exemplar-based image inpainting," *IEEE Trans. Image Process.*, Vol. 13, no. 9, pp. 1200–1212, Sep. 2004.

- [3] C. Cho and S. Lee, "Effective five directional partial derivatives-based image smoothing and a parallel structure design," *IEEE Trans. Image Process.*, vol. 25, no. 4, pp. 1617–1625, Apr. 2016.
- [4] X. Fu, J. Huang, X. Ding, Y. Liao, and J. Paisley, "Clearing the skies: A deep network architecture for single-image rain removal," *IEEE Trans. Image Process.*, vol. 26, no. 6, pp. 2944–2956, Jun. 2017.
- [5] S. D. Rane, G. Sapiro, M. Bertalmio, "Structure and texture filling-in of missing image blocks in wireless transmission and compression applications," *IEEE Trans. Image Process.*, vol. 12, no. 3, pp. 296–303, Mar. 2003. [6] J. Wu, W. Kong, J. Mielikainen, and B. Huang, "Lossless compression of hyperspectral imagery via clustered differential pulse code modulation with removal of focal spectral outliers," *IEEE Signal Process. Lett.*, vol. 22, no. 12, pp. 2194–2198, Dec. 2015.
- [7] D. Banerjee, K. Yu, and G. Aggarwal, "Robotic arm based 3D reconstruction test automation," *IEEE Access*, vol. 6, pp. 7206–7213, 2018.
- [8] D. Banerjee, K. Yu, and G. Aggarwal, "Hand jitter reduction algorithm software test automation using robotic Arm," *IEEE Access*, vol. 6, pp. 23582–23590, 2018, doi: [10.1109/ACCESS.2018.2829466](https://doi.org/10.1109/ACCESS.2018.2829466).
- [9] D. Allman, A. Reiter, and M. A. L. Bell, "Photoacoustic source detection and reflection artifact removal enabled by deep learning," *IEEE Trans. Med. Imag.*, vol. 37, no. 6, pp. 1464–1477, Jun. 2018.
- [10] D. Banerjee, K. Yu, and G. Aggarwal, "Image rectification software test automation using a robotic Arm," *IEEE Access*, vol. 6, pp. 34075–34085, 2018.
- [11] D. Banerjee and K. Yu, "Robotic Arm-based face recognition software test automation," *IEEE Access*, vol. 6, pp. 37858–37868, 2018.
- [12] J. A. Hashem, M. Pryor, S. Landsberger, J. Hunter, and D. R. Janecky, "Automating High-precision X-ray and neutron imaging applications with robotics," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 663–674, Apr. 2018.
- [13] Y. Tian, X. Li, K. Wang, and F.-Y. Wang, "Training and testing object detectors with virtual images," *IEEE/CAA J. Autom. Sinica*, vol. 5, no. 2, pp. 539–546, Mar. 2018.
- [14] J. Schneider, C. Stachniss, and W. Forstner, "On the accuracy of dense fisheye stereo," *IEEE Robot. Autom. Lett.*, vol. 1, no. 1, pp. 227–234, Jan. 2016.
- [15] J. Harel, C. Koch, and P. Perona, "Graph-Based Visual Saliency," in *Proc. Neural Inf. Process. Syst. (NIPS)*, 2006.
- [16] J. Harel, *A Saliency Implementation in MATLAB*. Accessed: Oct. 10, 2019. [Online]. Available: <http://www.klab.caltech.edu/~harel/share/gbvs.php>
- [17] X. Hou, J. Harel, and C. Koch, "Image signature: Highlighting sparse salient regions," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 1, pp. 194–201, Jan. 2012.
- [18] L. Fei-Fei, R. Fergus, and P. Perona, "Learning generative visual models from few training examples: An incremental Bayesian approach tested on 101 object categories," in *Proc. Conf. Comput. Vis. Pattern Recognit. Workshop*, Jun./Jul. 2004.



KEVIN YU has contributed to test automation validation for continuous integration and regression tests for computer vision algorithms. He has also validated computer vision engine features such as image rectification for Android software products. He is currently a Senior Test Engineer with Qualcomm Technologies, Inc., USA.



GARIMA AGGARWAL has actively worked on MATLAB postprocessing modules for CV features and various other automation projects. She is currently a Senior Test Engineer with Qualcomm Technologies, Inc., USA.

• • •



DEBDEEP BANERJEE (Member, IEEE) received the master's degree in electrical engineering from the Illinois Institute of Technology. He has more than ten years of industry experience in the field of software/systems engineering. He is currently a Senior Staff Engineer and the Engineering Manager with Qualcomm Technologies, Inc., USA. He is also the Software/Systems Development Engineer Test Lead of the computer vision project and is responsible for test

automation design, planning, development, deployment, code reviews, and project management. He also works closely with the software/system teams. He has been working with the Software Test Automation Team since the inception of the computer vision project at Qualcomm and also involved in managing and developing the robotic arm software used in the Computer Vision Laboratory.